

Software Requirements Specification

for

Health Management System

Prepared by:

Salma Khaled 192400687

Mariam Farouk 192400628

Jasmine Ahmed 192400649

Group 3-2

May 2026

Table of Contents

Table of Contents.....	2-3
1. Introduction	4
1.1 Purpose	4
1.2 Document Conventions	4
1.3 Intended Audience and Reading Suggestions	4
1.4 Product Scope	4
1.5 References	5
2. Overall Description.....	5
2.1 Product Perspective	5
2.2 Product Functions.....	5
2.3 User Classes and Characteristics	5
2.4 Operating Environment.....	6
2.5 Design and Implementation Constraints	6
2.6 User Documentation	6
2.7 Assumptions and Dependencies	6
3. External Interface Requirements	7
3.1 User Interfaces	7
3.2 Hardware Interfaces	7
3.3 Software Interfaces.....	7
3.4 Communications Interfaces	7
4. System Features	8
4.1 Landing Page & Navigation	8
4.1.1 Description and Priority	8
4.1.2 Stimulus/Response Sequences.....	8
4.1.3 Functional Requirements	8
4.2 Hospital Module.....	8
4.2.1 Description and Priority	8
4.2.2 Stimulus/Response Sequences.....	8
4.2.3 Functional Requirements	9
4.3 Pharmacy Module.....	9
4.3.1 Description and Priority	9
4.3.2 Stimulus/Response Sequences.....	9
4.3.3 Functional Requirements	10
4.4 Patient Portal.....	10

4.4.1 Description and Priority	10
4.4.2 Stimulus/Response Sequences.....	10
4.4.3 Functional Requirements	10
4.5 Data Persistence	11
4.5.1 Description and Priority	11
4.5.2 Stimulus/Response Sequences.....	11
4.5.3 Functional Requirements	11
5. Other Non-Functional Requirements.....	11
5.1 Performance Requirements	11
5.2 Safety Requirements	11
5.3 Security Requirements	11
5.4 Software Quality Attributes	11

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) describes the functional and non-functional requirements for the Health Management System. The project is a Qt/C++ desktop application designed to manage hospital patient workflows and pharmacy prescription workflows within a single unified GUI application.

This project is intended to serve as the authorization for medication across hospitals and pharmacies.

1.2 Document Convention

The following conventions are used throughout this document:

- Bold text indicates a key term being defined for the first time.
- Requirement identifiers follow the pattern REQ-XX-N, where XX is an abbreviated module code and N is a sequence number.
- Priority levels are designated as High, Medium, or Low.
- Higher-level requirements implicitly convey their priority to sub-requirements unless otherwise specified.

1.3 Intended Audience and Reading Suggestions

This document is intended for the following audiences:

- Developers: Read all sections in sequence. Pay particular attention to Sections 3 and 4 for interface and feature details.
- Testers / QA: Focus on Section 4 (System Features) and Section 5 (Non-functional Requirements) to derive test cases.
- Academic Evaluators / Instructors: The Introduction and Overall Description sections provide a concise project overview. Section 4 details the implemented features.
- End Users: Section 3.1 (User Interfaces) provides an overview of how to interact with the system.

1.4 Product Scope

The Patient Management System is a standalone Windows desktop application built with C++17 and the Qt 6 framework. It provides three role-based modules accessible from a central landing page:

- Hospital Module — allows hospital staff to check patients in and out, and view patient records.
- Pharmacy Module — allows pharmacists to add prescriptions and mark them as dispensed when a patient picks them up.
- Patient Portal — allows patients to self-serve and view their own hospital visit history and prescription records.

The system persists all data locally to a JSON file on the host machine. It does not connect to any network or external database. The project aims to reduce paperwork, centralise patient and prescription data, and streamline hospital and pharmacy workflows.

1.5 References

- Qt 6 Documentation — <https://doc.qt.io/qt-6/>
- CMake Documentation — <https://cmake.org/documentation/>
- IEEE SRS standards — <https://ieeexplore.ieee.org/document/278253>

2. Overall Description

2.1 Product Perspective

The Patient Management System is a new, self-contained desktop application. It is designed as a standalone tool that unifies hospital check-in/check-out, prescription management, and a patient self-service portal in one application window.

The application uses a QStackedWidget as its top-level navigation structure and role-specific QTabWidget containers for each module. All data is shared between modules through a singleton DataStore class, which serialises to and deserialises from a local JSON file on application start-up and shutdown.

2.2 Product Functions

At a high level, the project provides the following major functions:

- Landing page with role selection (Hospital Staff, Pharmacist, Patient).
- Role-based login authentication for Hospital and Pharmacy modules.
- Hospital module: patient check-in, patient check-out with diagnosis and medication recording, and a searchable data table of all patients.
- Pharmacy module: adding new prescriptions and processing prescription pick-ups.
- Patient portal: self-service login by name and ID to view personal hospital and prescription records.
- Automatic data persistence via JSON serialisation on application close.

2.3 User Classes and Characteristics

Three distinct user classes interact with the system:

- Hospital Staff — Trained clinical or administrative personnel who check patients in and out. They are expected to have basic computer literacy. They interact with the Hospital Module.
- Pharmacists — Healthcare professionals who manage prescriptions. They are expected to be familiar with medical terminology. They interact with the Pharmacy Module.
- Patients — End users who access the Patient Portal to view their own records. Minimal technical literacy is assumed; the portal interface is kept simple and intuitive.

2.4 Operating Environment

The project operates in the following environment:

- Operating System: Windows 10 or Windows 11 (64-bit).
- Framework: Qt 6.10.1 (Widgets module, Core module).
- Compiler: MinGW 64-bit (GCC-based toolchain bundled with Qt).
- Build System: CMake 3.16 or later.
- Development IDE: Qt Creator.
- Storage: Local filesystem — data is stored in the application data directory (QStandardPaths::AppDataLocation) as data.json.

2.5 Design and Implementation Constraints

- The application must be implemented entirely in C++17 using the Qt 6 Widgets framework.
- No external database or network connectivity is permitted; all persistence must use local JSON files via Qt's QJsonDocument API.
- The project must be buildable with CMake and the MinGW toolchain on Windows.
- Authentication credentials are hardcoded in the source code (acceptable for academic scope); a production system would require a secure credential store.
- All UI components must be constructed programmatically; Qt Designer is not used.

2.6 User Documentation

The following user documentation will be delivered with the software:

- This SRS document, which serves as a technical reference.
- Inline placeholder text (QLineEdit::setPlaceholderText) within the application to guide data entry.
- Informational labels within the UI explaining key behaviours

2.7 Assumptions and Dependencies

- It is assumed that users access the application on a single machine; multi-user simultaneous access is not supported.
- The application depends on the Qt Core and Qt Widgets modules. No third-party libraries beyond Qt are required.
- Data integrity is not guaranteed if the process is terminated abnormally (e.g., power failure), as save() is called only on clean application exit.

3. External Interface Requirements

3.1 User Interfaces

The project provides a fully graphical Qt Widgets-based desktop interface. The following UI principles are applied throughout:

- A landing page presents three clearly labelled role-selection buttons (Hospital, Pharmacy, Patient).
- Hospital and Pharmacy modules require credential entry via a shared LoginPage widget before access is granted.
- Each module uses a QTabWidget to separate its sub-functions (e.g., Check In / Check Out / Patient Data for the Hospital module).
- All data-entry forms use QLineEdit and QTextEdit fields with descriptive placeholder text.
- Status labels and QMessageBox dialogs provide real-time feedback on successful or failed operations.
- The Patient Portal uses a QStackedWidget to switch between a login screen and a records screen.
- Colour scheme: dark blue (#1F4E79) for headings, medium blue (#2E75B6) for accents, white backgrounds for form areas.
- Font size for titles: 20px bold; standard form labels use Qt defaults.

3.2 Hardware Interfaces

The project does not interface directly with any hardware components beyond the standard PC peripherals (keyboard, mouse, display). No specialist hardware such as barcode scanners, printers, or card readers is required or supported in this version.

3.3 Software Interfaces

The project interfaces with the following software components:

- Qt 6 Core Module (Qt6::Core) — Provides QString, QList, QJsonDocument, QJsonObject, QJsonArray, QFile, QDir, and QStandardPaths.
- Qt 6 Widgets Module (Qt6::Widgets) — Provides all UI classes: QMainWindow, QWidget, QStackedWidget, QTabWidget, QLineEdit, QTextEdit, QLabel, QPushButton, QTableWidget, QFormLayout, QVBoxLayout.
- Local filesystem — The application reads and writes a single JSON file (data.json) in the OS application data directory.

3.4 Communications Interfaces

This version of the project All data exchange is local to the host machine.

4. System Features

4.1 Landing Page & Navigation

4.1.1 Description and Priority

The landing page is the entry point of the application. It presents three role-selection buttons that navigate to the appropriate module. Priority: High.

4.1.2 Stimulus/Response Sequences

- User launches the application → Landing page is displayed.
- User clicks "Hospital" → HospitalLogin page is displayed.
- User clicks "Pharmacy" → PharmacyLogin page is displayed.
- User clicks "Patient" → Patient Portal is displayed.
- User clicks "Back" on any login page → Landing page is restored.

4.1.3 Functional Requirements

REQ ID	Description	Priority
REQ-1	The system shall display a landing page with three role-selection buttons on application start.	High
REQ-2	Clicking 'Hospital' shall navigate to the Hospital login page.	High
REQ-3	Clicking 'Pharmacy' shall navigate to the Pharmacy login page.	High
REQ-4	Clicking 'Patient' shall navigate to the Patient Portal.	High
REQ-5	Each login page shall provide a 'Back' button that returns the user to the landing page.	Medium

4.2 Hospital Module

4.2.1 Description and Priority

The Hospital Module is a tabbed widget with three tabs: Check In, Check Out, and Patient Data. It allows hospital staff to manage the full lifecycle of a patient visit. Priority: High.

4.2.2 Stimulus/Response Sequences

- Staff selects 'Check In' tab → A form is field with Patient Name, Patient ID, Doctor Name, and Specialization is displayed.
- Staff fills in all fields and clicks 'Check In' → A new Patient record is added to the DataStore; a success status message is shown; the form is cleared.
- Staff selects 'Check Out' tab → A form with Patient Name, Patient ID, Diagnosis, and Medication fields is displayed.
- Staff enters a valid Patient ID and clicks 'Check Out' → The patient's record is updated with diagnosis and medication; the patient is marked as checkedOut; a success message is shown.

- Staff selects 'Patient Data' tab → A table showing all registered patients and their details is displayed.
- Staff enters an ID in the search box and clicks 'Search' → The table is filtered to show only the matching patient.
- Staff clicks 'Clear' → The full table is restored.

4.2.3 Functional Requirements

REQ ID	Description	Priority
REQ-6	The Hospital Module shall provide a Check In tab for registering new patients.	High
REQ-7	Check-in form shall collect: Patient Name, Patient ID, Doctor Name, and Doctor Specialization. The patient record shall be added to the DataStore.	High
REQ-8	Check-out form shall collect: Patient Name, Patient ID, Diagnosis, Medication notes.	High
REQ-9	The Hospital Module shall provide a Patient Data tab with a table of all patients. Display columns for: Patient ID, Name, Doctor, Specialization, Diagnosis, Status.	High
REQ-10	Staff shall be able to search the Patient Data table by Patient ID.	Medium
REQ-11	A 'Clear' button shall restore the full patient table after a search.	Low
REQ-12	The Patient Data table shall refresh automatically after check-in or check-out events.	High

4.3 Pharmacy Module

4.3.1 Description and Priority

The Pharmacy Module is a tabbed widget with two tabs: Add Prescription and Pickup Prescription.

4.3.2 Stimulus/Response Sequences

- Pharmacist selects 'Add Prescription' tab → A form with Patient Name, Patient ID, Pharmacist Name, Diagnosis, and Medication fields is displayed.
- Pharmacist fills in all fields and clicks 'Add Prescription' → A Prescription record (taken = true) is added to the DataStore; a success message is shown; the form is cleared.
- Pharmacist selects 'Pickup Prescription' tab → A form to look up pending prescriptions by Patient ID is displayed.
- Pharmacist enters Patient ID and clicks 'Look Up' → All pending prescriptions for that patient are listed.
- Pharmacist clicks 'Mark as Dispensed' → The prescription's taken flag is set to true; the prescription is removed from the pending list.

4.3.3 Functional Requirements

REQ ID	Description	Priority
REQ-13	Add Prescription form shall collect: Patient Name, Patient ID, Pharmacist Name, Diagnosis, Medication. [taken = true (already dispensed)].	High
REQ-14	The system shall prevent submission if any field is empty, displaying a warning dialog.	High
REQ-15	The Pickup Prescription tab, the user gives an ID to show list all pending prescriptions (taken = false) for a given Patient ID.	High
REQ-16	Pharmacists shall be able to mark individual prescriptions as dispensed (taken = true).	High
REQ-17	The pending prescription list shall update immediately after a prescription is dispensed.	Medium

4.4 Patient Portal

4.4.1 Description and Priority

The Patient Portal allows patients to log in using their name and Patient ID and view their own hospital records and prescription history.

4.4.2 Stimulus/Response Sequences

- Patient clicks 'Patient' on the landing page → Patient Portal login screen is displayed.
- Patient enters Name and Patient ID and clicks 'View My Records' → The system searches the DataStore; if found, the records screen is displayed with their hospital and prescription history.
- Patient clicks 'Logout' → The login screen is restored.

4.4.3 Functional Requirements

REQ ID	Description	Priority
REQ-18	The Patient Portal shall display a login screen with Name and Patient ID fields.	High
REQ-19	The system shall authenticate the patient by matching Name and ID against the DataStore.	High
REQ-20	On successful login, the system shall display the patient's prescription history in a separate table.	High
REQ-21	On failed login, the system shall display an inline error message.	Medium
REQ-22	A 'Logout' button shall return the patient to the portal login screen.	Low

4.5 Data Persistence

4.5.1 Description and Priority

All patient and prescription data must be persisted across application sessions using a local JSON file.

4.5.2 Stimulus/Response Sequences

- Application starts → `DataStore::load()` reads `data.json` from `AppDataLocation`; patients and prescriptions are populated.
- Application exits normally → `DataStore::save()` serialises all patients and prescriptions to `data.json`.

4.5.3 Functional Requirements

REQ ID	Description	Priority
REQ-23	The system shall load and save all patient and prescription data from a local JSON file on start-up.	High
REQ-24	The JSON file shall be stored in <code>QStandardPaths::AppDataLocation</code> .	High
REQ-25	The system shall create the storage directory if it does not exist before writing.	Medium
REQ-26	The <code>DataStore</code> shall be a singleton accessible from all modules.	High

5. Other Non-Functional Requirements

5.1 Performance Requirements

The application shall launch and display the landing page within 3 seconds on a modern PC.

All switching between taps shall complete within 200 milliseconds.

Data load on start-up and save on exit shall complete within 1 second for up to 10,000 patient records and 10,000 prescription records.

5.2 Safety Requirements

The project stores healthcare-related data. All data is stored immediately with no delay or loss possible.

5.3 Security Requirements

All pages have a login that only the staff should know to have more security.

5.4 Software Quality Attributes

The interface uses clear labels, placeholder text, and status messages to guide users with minimal training. All forms shall provide inline validation feedback.